

INSTITUCIÓN UNIVERSITARIA ITM

Facultad de Ingeniería

Programa de Ingeniería de Sistemas

INFORME ACADÉMICO

Asistente Académico Básico con API de Google Gemini

Estudiante: Nombre completo aquí

Asignatura: Herramientas de Inteligencia Artificial

Docente: Julio Pastor Restrepo Zapata

Fecha: 3 de abril de 2026

Ciudad: Medellín, Colombia

Índice

Resumen	3
1. Introducción	4
2. Objetivos	4
2.1. Objetivo general	4
2.2. Objetivos específicos	4
3. Descripción de la herramienta utilizada	5
4. Desarrollo de la práctica	5
4.1. Preparación del entorno	5
4.2. Configuración de la API Key	5
4.3. Instalación de dependencias	6
4.4. Construcción del script	6
5. Resultados obtenidos en las pruebas	6
5.1. Prueba 1: solicitud académica	7
5.2. Prueba 2: excusa o justificación	7
5.3. Prueba 3: texto informativo	7
5.4. Capturas de evidencia	7
6. Análisis crítico de los resultados	9
6.1. ¿Qué tan coherente fue el resultado?	9
6.2. ¿Hubo ambigüedades?	10
6.3. ¿La clasificación siempre fue correcta?	10
6.4. ¿Qué pasaría con mensajes más complejos?	10

6.5. ¿Qué limitaciones se observaron?	11
6.6. ¿Cómo se podría mejorar el prompt?	11
6.7. ¿Qué riesgos éticos existen?	12
7. Preguntas orientadoras	12
7.1. ¿Qué es una API y por qué resulta importante en el desarrollo de aplicaciones con inteligencia artificial?	12
7.2. ¿Qué diferencias existen entre usar un chat de IA y consumir un modelo mediante API?	12
7.3. ¿Cómo influyó la redacción del prompt en la calidad de la respuesta obtenida?	13
7.4. ¿Qué limitaciones observó en las respuestas generadas por el sistema?	13
7.5. ¿Qué mejoras implementaría si este prototipo fuera a utilizarse en un entorno real?	13
7.6. ¿Qué consideraciones éticas deben tenerse en cuenta al usar inteligencia artificial para apoyar la comunicación académica?	13
8. Conclusiones	14
9. Anexo: código fuente	15
10. Referencias	21

Resumen

En este trabajo se desarrolló un asistente académico básico en Python, conectado a la API de Google Gemini mediante la librería `google-genai`. El propósito principal fue aplicar, de manera práctica, conceptos relacionados con el consumo de una API, el diseño de prompts y el análisis crítico de respuestas generadas por inteligencia artificial.

El sistema implementado permite procesar un texto de entrada y ejecutar tres tareas: resumir el mensaje, clasificarlo según su intención y redactar una respuesta formal adecuada al contexto académico. Durante la práctica se realizaron varias pruebas funcionales con mensajes de solicitud, excusa e información técnica, obteniendo resultados coherentes y útiles en la mayoría de los casos.

Además de la implementación técnica, el informe incluye un análisis reflexivo sobre la calidad de las respuestas, las limitaciones observadas, las mejoras posibles al prompt y algunas consideraciones éticas asociadas al uso de inteligencia artificial en escenarios académicos.

1. Introducción

La inteligencia artificial ha dejado de ser una tecnología lejana o exclusiva de laboratorios especializados para convertirse en una herramienta de uso cotidiano en diferentes áreas del conocimiento. En el contexto académico, su aplicación ha crecido de forma notable, no solo como apoyo en la generación de contenido, sino también como recurso para automatizar tareas, organizar información y agilizar procesos de comunicación.

Sin embargo, el uso responsable de estas herramientas exige más que saber interactuar con un chat. También implica comprender cómo se integran los modelos de inteligencia artificial en aplicaciones reales mediante API, cómo se construyen instrucciones claras para obtener mejores respuestas y cómo se evalúa críticamente la salida generada por el sistema.

En ese sentido, el presente trabajo se desarrolló a partir del taller propuesto en la asignatura *Herramientas de Inteligencia Artificial*, cuyo objetivo fue construir un asistente académico básico usando una API gratuita de IA. El ejercicio permitió pasar de un uso meramente intuitivo de la inteligencia artificial a una aplicación más técnica, estructurada y reflexiva. Según el enunciado del taller, el sistema debía ejecutar tres tareas principales sobre un texto de entrada: resumen, clasificación y redacción formal. :contentReference[oaicite:0]index=0

2. Objetivos

2.1. Objetivo general

Desarrollar un asistente académico básico mediante el uso de una API gratuita de inteligencia artificial, con el fin de aplicar conceptos fundamentales de integración tecnológica, diseño de instrucciones y análisis crítico de resultados.

2.2. Objetivos específicos

- Comprender el concepto de API y su papel dentro de la integración de servicios de inteligencia artificial.
- Implementar un script funcional en Python que permita enviar instrucciones a un modelo de IA y recibir respuestas generadas automáticamente.

- Diseñar prompts orientados a tareas específicas como resumen, clasificación y redacción formal.
- Evaluar la pertinencia, claridad y utilidad de las respuestas generadas por la herramienta.
- Reflexionar sobre las oportunidades, limitaciones y buenas prácticas asociadas al uso de la inteligencia artificial en entornos académicos.

3. Descripción de la herramienta utilizada

Para el desarrollo del taller se utilizó la API de Google Gemini, integrada en Python mediante la librería `google-genai`. El modelo empleado fue `gemini-2.5-flash`, el cual permitió realizar tareas de generación de texto de forma rápida y consistente. El código base del taller también proponía el uso de la variable de entorno `GEMINI_API_KEY`, lo cual representa una buena práctica de seguridad.

```
contentReference[oaicite : 1]index = 1
```

Desde el punto de vista técnico, la herramienta resulta útil porque permite consumir un servicio de IA desde un script, sin depender únicamente de una interfaz conversacional. Esto abre la puerta a automatizaciones, integraciones con otras aplicaciones y flujos de trabajo más controlados.

4. Desarrollo de la práctica

4.1. Preparación del entorno

Antes de ejecutar el programa, se preparó un entorno virtual en Python para evitar conflictos entre dependencias. Luego se instaló la librería necesaria para comunicarse con la API de Gemini. Este paso fue importante porque permitió trabajar en un entorno limpio y reproducible.

4.2. Configuración de la API Key

La autenticación se realizó mediante la variable de entorno `GEMINI_API_KEY`. Esta decisión fue una buena práctica de seguridad para cualquier proyecto que use APIs.

4.3. Instalación de dependencias

Se instaló la librería oficial para acceder al servicio de Gemini desde Python. Con esto, el script quedó listo para inicializar el cliente y enviar solicitudes al modelo.

4.4. Construcción del script

El programa desarrollado realiza las siguientes acciones:

1. Lee la API Key desde la variable de entorno.
2. Inicializa el cliente de Gemini.
3. Solicita al usuario que ingrese o seleccione un texto.
4. Construye tres prompts independientes: resumen, clasificación y respuesta formal.
5. Envía cada prompt al modelo.
6. Muestra los resultados en pantalla.
7. Guarda los resultados en un archivo de texto para su posterior revisión.

Este enfoque permitió organizar mejor el flujo del sistema y facilitó las pruebas funcionales. Además, el hecho de trabajar con prompts separados ayudó a obtener respuestas más específicas y ordenadas.

5. Resultados obtenidos en las pruebas

Durante la ejecución del programa se realizaron pruebas con distintos tipos de texto: una solicitud académica, una excusa por inasistencia y un mensaje informativo de carácter técnico. Los resultados obtenidos mostraron un comportamiento bastante coherente por parte del modelo.

5.1. Prueba 1: solicitud académica

En este caso, el mensaje indicaba una ausencia a clase por una cita médica y preguntaba si era posible entregar la actividad al día siguiente. El sistema resumió correctamente la idea principal, clasificó el mensaje como *solicitud* y redactó una respuesta formal adecuada.

5.2. Prueba 2: excusa o justificación

En la segunda prueba, el texto expresaba una imposibilidad para asistir a clase o entregar una actividad debido a una emergencia familiar. El modelo mantuvo una respuesta coherente, aunque la clasificación volvió a ubicar el mensaje como *solicitud*. Esto no necesariamente representa un error grave, ya que el mensaje no solo contiene una excusa, sino también una petición implícita de aplazamiento.

5.3. Prueba 3: texto informativo

En el tercer caso se utilizó un texto más técnico sobre el uso de APIs en inteligencia artificial. La respuesta fue bastante precisa: el resumen condensó bien las ideas centrales, la clasificación fue *informativo* y la respuesta formal conservó un tono académico.

5.4. Capturas de evidencia

En esta sección se recomienda insertar las capturas de pantalla de las pruebas realizadas.


```

C:\Windows\System32\cmd.exe
(venv) E:\Universidad\ITM\Materias\Ingeniería de Sistemas\3. Herramientas de Inteligencia Artificial\Entregables\Entrega 2\API_Google\Asistente-Acad-mico-B-sico>python
"2. Código/API_Google.py"

=== Asistente Académico Básico ===
1. Usar texto de solicitud académica
2. Usar texto de excusa
3. Usar texto informativo
4. Escribir mi propio texto

Elige una opción (1-4): 1

=====
RESUMEN:
Un estudiante informa que faltó a clase por una cita médica y pregunta si es posible entregar la actividad mañana.
CLASIFICACIÓN:
solicitud
RESPUESTA FORMAL:
Estimado/a estudiante,

Agradezco que me haya informado sobre su situación. Sí, puede entregar la actividad pendiente mañana.

Saludos cordiales.
=====
Resultados guardados en: 4. Análisis crítico\resultado_asistente_20260403_014508.txt

```

Figura 1: Caso 1: Ejecución del sistema con el caso de solicitud académica

```

C:\Windows\System32\cmd.exe
(venv) E:\Universidad\ITM\Materias\Ingeniería de Sistemas\3. Herramientas de Inteligencia Artificial\Entregables\Entrega 2\API_Google\Asistente-Acad-mico-B-sico>python
"2. Código/API_Google.py"

=== Asistente Académico Básico ===
1. Usar texto de solicitud académica
2. Usar texto de excusa
3. Usar texto informativo
4. Escribir mi propio texto

Elige una opción (1-4): 2

=====
RESUMEN:
El remitente solicita permiso para entregar un trabajo con retraso debido a una emergencia familiar.
CLASIFICACIÓN:
solicitud
RESPUESTA FORMAL:
Estimado/a estudiante,

Comprendo la situación de emergencia familiar que nos ha comunicado.
Agradezco que nos haya informado. Puede enviar su trabajo a la brevedad posible.

Saludos cordiales.
=====
Resultados guardados en: 4. Análisis crítico\resultado_asistente_20260403_014748.txt

```

Figura 2: Caso 2: Ejecución del sistema con el caso de excusa

```

C:\Windows\System32\cmd.exe
(venv) E:\Universidad\ITM\Materias\Ingeniería de Sistemas\3. Herramientas de Inteligencia Artificial\Entregables\Entrega 2\API_Google\Asistente-Acad-mico-B-sico>python
"2. Código/API_Google.py"

=== Asistente Académico Básico ===
1. Usar texto de solicitud académica
2. Usar texto de excusa
3. Usar texto informativo
4. Escribir mi propio texto

Elige una opción (1-4): 3

=====
RESUMEN:
La inteligencia artificial mediante APIs automatiza tareas de procesamiento de texto, integración de servicios y generación de respuestas, requiriendo manejo de credenciales, diseño de prompts y validación de resultados para asegurar su calidad.
CLASIFICACIÓN:
informativo
RESPUESTA FORMAL:
Comprendemos la significativa capacidad de la inteligencia artificial, a través de APIs, para automatizar tareas clave de procesamiento y generación de respuestas. Coincidimos en que su despliegue efectivo requiere un riguroso manejo de credenciales, un diseño cuidadoso de prompts y una validación constante para garantizar la calidad y pertinencia esperadas.
=====
Resultados guardados en: 4. Análisis crítico\resultado_asistente_20260403_014816.txt
(venv) E:\Universidad\ITM\Materias\Ingeniería de Sistemas\3. Herramientas de Inteligencia Artificial\Entregables\Entrega 2\API_Google\Asistente-Acad-mico-B-sico>

```

Figura 3: Caso 3: Ejecución del sistema con el caso informativo

```
C:\Windows\System32\cmd.exe
(venv) E:\Universidad\ITM\Materias\Ingeniería de Sistemas\3. Herramientas de Inteligencia Artificial\Entregables\Entrega 2\API_Google\Asistente-Acad-mico-B-sico>python
"2. Código/API_Google.py"

=== Asistente Académico Básico ===
1. Usar texto de solicitud académica
2. Usar texto de excusa
3. Usar texto informativo
4. Escribir mi propio texto

Elige una opción (1-4):

Escribe tu texto. Finaliza con una línea vacía:
Buenos días profesor.
No pude asistir a clase porque estaba enfermo.
¿Puedo enviar la actividad mañana?
Gracias.

=====
RESUMEN:
El estudiante informa al profesor de su ausencia por enfermedad y solicita permiso para entregar la actividad mañana.
CLASIFICACIÓN:
solicitud
RESPUESTA FORMAL:
Estimado estudiante:
Lamento que su estado de salud le impidiera asistir a clase. Agradezco que me informe.
Sí, puede enviar la actividad pendiente mañana.
Saludos cordiales.
=====
Resultados guardados en: 4. Análisis crítico\resultado_asistente_20260403_014718.txt
```

Figura 4: Caso 4: Texto informativo técnico

6. Análisis crítico de los resultados

Esta es probablemente la parte más importante del trabajo, porque no se trata solamente de mostrar que el programa funciona, sino de analizar con criterio qué tan bien respondió la herramienta y cuáles son sus límites reales.

6.1. ¿Qué tan coherente fue el resultado?

En términos generales, el resultado fue bastante coherente. El modelo entendió bien el propósito de cada prompt y respondió de forma alineada con el tipo de tarea solicitada. Los resúmenes conservaron la idea principal, la clasificación fue razonable y las respuestas formales mantuvieron un tono adecuado para un contexto académico.

Lo más positivo fue que el sistema no se desvió del objetivo, no inventó información innecesaria y conservó una estructura clara en la salida. Esto demuestra que, con prompts bien formulados, la inteligencia artificial puede convertirse en un apoyo útil y bastante confiable.

6.2. ¿Hubo ambigüedades?

Sí, hubo algunas ambigüedades, aunque no fueron críticas. El caso más claro apareció en mensajes que mezclaban excusa y solicitud al mismo tiempo. Por ejemplo, cuando un estudiante explica una razón para no asistir y luego pide entregar la actividad después, el mensaje puede interpretarse tanto como *excusa* como *solicitud*.

En estos casos, la clasificación depende de cuál intención predomine. Esto muestra que el modelo no siempre trabaja con categorías absolutamente cerradas, sino que interpreta el sentido general del mensaje. Por eso, si el texto tiene varios propósitos al mismo tiempo, la respuesta puede variar ligeramente.

6.3. ¿La clasificación siempre fue correcta?

No necesariamente en un sentido absoluto, pero sí fue razonable en función del contenido. En el caso de la excusa por enfermedad o emergencia familiar, el modelo la clasificó como *solicitud*. Aunque a primera vista podría parecer discutible, en realidad el mensaje también contenía una petición implícita de permiso o aplazamiento.

Esto demuestra que la clasificación automática no siempre debe evaluarse de forma rígida. En textos reales, la intención del mensaje suele ser mixta, y la IA puede tomar una decisión válida aunque no coincida exactamente con lo que una persona esperaría en una primera lectura.

6.4. ¿Qué pasaría con mensajes más complejos?

Con mensajes más complejos, el comportamiento podría volverse menos preciso. Si el texto incluye varias intenciones, sarcasmo, lenguaje muy informal, errores ortográficos graves o referencias contextuales muy específicas, el modelo podría resumir bien la idea general pero equivocarse al clasificarla o al redactar la respuesta.

También es posible que, ante textos demasiado largos o ambiguos, la IA simplifique demasiado la información. En ese caso, el resultado seguiría siendo útil, pero perdería detalles importantes. Por eso, para mensajes complejos, sería necesario mejorar el prompt, dividir la tarea en pasos más específicos o incluso agregar validaciones adicionales.

6.5. ¿Qué limitaciones se observaron?

A partir de las pruebas realizadas, se pueden identificar varias limitaciones:

- La salida depende mucho de la calidad del prompt.
- El sistema no tiene comprensión humana profunda del contexto.
- La clasificación puede variar cuando un texto tiene más de una intención.
- No existe verificación automática de veracidad.
- El modelo puede responder de forma correcta en forma, pero no necesariamente en fondo si el mensaje es ambiguo.

En otras palabras, la herramienta es útil, pero no reemplaza el criterio humano. Su valor está en apoyar la tarea, no en tomar decisiones finales por sí sola.

6.6. ¿Cómo se podría mejorar el prompt?

El prompt puede mejorarse bastante con pequeñas modificaciones. Algunas ideas serían:

- Definir mejor las categorías de clasificación.
- Pedir explícitamente que el modelo responda con una sola etiqueta.
- Indicar un límite máximo de líneas para la respuesta formal.
- Incluir ejemplos de entrada y salida.
- Establecer qué hacer cuando un mensaje tenga más de una intención.

También sería útil agregar instrucciones del tipo: “si el mensaje mezcla excusa y solicitud, clasifica según la intención dominante”. Eso reduce la ambigüedad y hace que el modelo se comporte de manera más estable.

6.7. ¿Qué riesgos éticos existen?

El uso de inteligencia artificial en entornos académicos también trae riesgos éticos importantes. Uno de los más evidentes es la posibilidad de usar la herramienta para reemplazar completamente el trabajo propio, lo cual afecta la honestidad académica.

Otro riesgo es la dependencia excesiva: si el estudiante se acostumbra a delegar todo en la IA, puede perder capacidad de análisis, redacción y criterio. También existe el tema de la privacidad, especialmente si se ingresan datos personales o sensibles en plataformas externas.

Por último, hay que tener presente que una IA puede sonar convincente incluso cuando se equivoca. Por eso, cualquier resultado debe ser revisado antes de ser usado como respuesta final.

7. Preguntas orientadoras

7.1. ¿Qué es una API y por qué resulta importante en el desarrollo de aplicaciones con inteligencia artificial?

Una API es una interfaz que permite que dos sistemas se comuniquen entre sí. En este caso, funciona como el puente entre el programa en Python y el modelo de inteligencia artificial. Su importancia radica en que permite integrar capacidades inteligentes dentro de aplicaciones reales sin tener que construir el modelo desde cero.

7.2. ¿Qué diferencias existen entre usar un chat de IA y consumir un modelo mediante API?

Usar un chat de IA es más directo y visual, mientras que consumir un modelo mediante API permite automatizar procesos, integrar la IA en sistemas propios y controlar mejor la lógica de funcionamiento. En otras palabras, el chat sirve para interactuar manualmente; la API sirve para construir soluciones.

7.3. ¿Cómo influyó la redacción del prompt en la calidad de la respuesta obtenida?

Influyó muchísimo. Cuando el prompt fue claro, corto y específico, las respuestas fueron más útiles y ordenadas. En cambio, si la instrucción fuera ambigua, el modelo podría responder con información más general o menos precisa. Esto confirma que un buen prompt es casi tan importante como el modelo mismo.

7.4. ¿Qué limitaciones observó en las respuestas generadas por el sistema?

La principal limitación fue la interpretación de mensajes con doble intención. También se notó que el modelo depende del contexto dado y que no siempre diferencia de manera perfecta entre excusa, solicitud o consulta. Aun así, su desempeño general fue bueno y funcional para el objetivo del taller.

7.5. ¿Qué mejoras implementaría si este prototipo fuera a utilizarse en un entorno real?

Se podría mejorar agregando una interfaz gráfica sencilla, un sistema de validación más robusto, opciones para guardar los resultados en formato PDF o Word y una lógica más detallada para clasificar mensajes complejos. También sería útil registrar el historial de consultas.

7.6. ¿Qué consideraciones éticas deben tenerse en cuenta al usar inteligencia artificial para apoyar la comunicación académica?

Es necesario usar la IA como apoyo y no como sustituto del trabajo propio. También se deben proteger las credenciales, evitar ingresar información sensible, revisar la calidad de las respuestas y mantener una postura responsable frente a la autoría y la honestidad académica.

8. Conclusiones

El desarrollo de este asistente académico básico permitió comprender de manera práctica cómo una API de inteligencia artificial puede integrarse en un programa funcional en Python. Más allá de ejecutar un código, la experiencia permitió aprender sobre autenticación, consumo de servicios, diseño de prompts y análisis de resultados.

Uno de los hallazgos más importantes fue comprobar que la calidad de la salida depende en gran medida de la claridad de la instrucción. Cuando el prompt está bien formulado, el modelo responde de forma más precisa, coherente y útil. Sin embargo, también quedó claro que la IA no interpreta todo con exactitud absoluta y que todavía necesita supervisión humana, sobre todo en mensajes ambiguos o complejos.

En términos generales, el trabajo cumplió con el objetivo planteado en el taller y dejó un aprendizaje valioso: la inteligencia artificial puede ser una herramienta poderosa en el entorno académico, pero su uso correcto exige criterio, responsabilidad y una buena base técnica.

9. Anexo: código fuente

El código fuente completo se encuentra disponible públicamente en el repositorio oficial del proyecto en GitHub:

<https://github.com/D13G04L3X/Asistente-Acad-mico-B-sico.git>

No obstante, se presenta el código utilizado para la implementación del asistente. Se recomienda incluir aquí la versión final del archivo API_Google.py.

```
import os
from pathlib import Path
from datetime import datetime
from google import genai

MODEL = "gemini-2.5-flash"

SAMPLE_TEXTS = {
    "1": """
Buenas tardes profesor.

No pude asistir a clase porque me encontraba en una cita médica.

Quisiera saber si es posible entregar la actividad mañana.

Gracias.
""",
    "2": """
Buenos días.

No pude presentar el trabajo en la fecha acordada porque tuve una
emergencia familiar.

Agradezco si me permite enviarlo con retraso.

Cordial saludo.
""",
    "3": """
La inteligencia artificial aplicada mediante APIs permite automatizar
```



```
    tareas de procesamiento de texto,
    integraci n de servicios y generaci n de respuestas. Su uso
    requiere manejo de credenciales, dise o de prompts
    y validaci n de resultados para asegurar calidad y pertinencia.
    """
}

def get_client() -> genai.Client:
    api_key = os.getenv("GEMINI_API_KEY")
    if not api_key:
        raise EnvironmentError(
            "No se encontr la variable de entorno GEMINI_API_KEY."
        )
    return genai.Client(api_key=api_key)

def ask_model(client: genai.Client, prompt: str) -> str:
    response = client.models.generate_content(
        model=MODEL,
        contents=prompt
    )
    text = getattr(response, "text", None)
    return text.strip() if text else "No se recibí una respuesta de texto."

def summarize_text(client: genai.Client, text: str) -> str:
    prompt = f"""
Eres un asistente acad mico.

Resume el siguiente texto en UNA sola oraci n, clara y precisa.
No agregues informaci n nueva. No expliques el proceso. Responde
solo con el resumen.

Texto:
{text}
"""
```

```
    return ask_model(client, prompt)

def classify_text(client: genai.Client, text: str) -> str:
    prompt = f"""
Eres un asistente académico.

Clasifica el siguiente mensaje en UNA sola categoría, usando
exactamente una de estas opciones:
- solicitud
- excusa
- agradecimiento
- queja
- consulta
- informativo
- otro

Responde solo con la categoría elegida. No agregues explicación.

Texto:
{text}
"""
    return ask_model(client, prompt)

def formal_reply(client: genai.Client, text: str) -> str:
    prompt = f"""
Eres un asistente académico.

Redacta una respuesta formal, breve, respetuosa y coherente para el
siguiente mensaje.
Extensión sugerida: 3 a 5 líneas.
No uses listas. No expliques el proceso. Solo escribe la respuesta.

Mensaje:
{text}
"""
    return ask_model(client, prompt)
```

```
def get_user_text() -> str:
    print("\n=== Asistente Acad mico B sico ===")
    print("1. Usar texto de solicitud acad mica")
    print("2. Usar texto de excusa")
    print("3. Usar texto informativo")
    print("4. Escribir mi propio texto")

    option = input("\nElige una opci n (1-4): ").strip()

    if option in SAMPLE_TEXTS:
        return SAMPLE_TEXTS[option].strip()

    print("\nEscribe tu texto. Finaliza con una l nea vac a:")
    lines = []
    while True:
        line = input()
        if line.strip() == "":
            break
        lines.append(line)
    return "\n".join(lines).strip()

def save_results(original: str, summary: str, classification: str,
    reply: str) -> Path:
    output_dir = Path("4. An lisis cr tico")
    output_dir.mkdir(parents=True, exist_ok=True)

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    output_file = output_dir / f"resultado_asistente_{timestamp}.txt"

    content = f"""ASISTENTE ACAD MICO B SICO
Fecha: {datetime.now().strftime("%Y-%m-%d %H:%M:%S")}

TEXTO ORIGINAL:
{original}
```

```
RESUMEN:
{summary}

CLASIFICACIÓN:
{classification}

RESPUESTA FORMAL:
{reply}
"""

    output_file.write_text(content, encoding="utf-8")
    return output_file

def main():
    try:
        client = get_client()
        user_text = get_user_text()

        if not user_text:
            print("No ingresaste texto. El programa finaliz .")
            return

        summary = summarize_text(client, user_text)
        classification = classify_text(client, user_text)
        reply = formal_reply(client, user_text)

        print("\n" + "=" * 60)
        print("RESUMEN:")
        print(summary)

        print("\nCLASIFICACIÓN:")
        print(classification)

        print("\nRESPUESTA FORMAL:")
        print(reply)
        print("=" * 60)
```

```
        saved_file = save_results(user_text, summary, classification,
                                   reply)
        print(f"\nResultados guardados en: {saved_file}")

    except Exception as e:
        print(f"\nOcurrió un error: {e}")

if __name__ == "__main__":
    main()
```

10. Referencias

- Institución Universitaria ITM. *Taller Práctico: Uso de una API Gratuita de Inteligencia Artificial para la Construcción de un Asistente Académico Básico.*
- Documentación oficial de Google Gemini y librería google-genai.